

Fiche descriptive de projet Hub

Kairos

Contexte et but du projet

Détailler l'origine du projet, les éléments motivants sa réalisation et un descriptif de l'usage final de celui-ci

Afin d'**apprendre le langage Lua** et le framework de jeu **LÖVE (Love2D)**, je souhaite créer "Kairos", un jeu vidéo 2D de survie et de gestion de ressources se déroulant sur la Lune. Je n'ai jamais pratiqué Lua : ce projet est l'occasion de découvrir le langage sur un cas concret, en construisant un petit jeu complet plutôt qu'une suite d'exercices.

Usage final : un jeu de survie sur PC, où le joueur, bloqué sur la Lune, doit réparer sa fusée endommagée afin de regagner la Terre le plus rapidement possible.

Porteur(s) du projet

Détailler ici les membres du groupe projet, leur rôle si défini ainsi que les éventuels partenaires externes [entreprise, communauté open-source, ...] et leur rôle.

- **Marc-Alexandre Baehr** — Étudiant en 2e année, porteur unique du projet : conception, développement, interface, équilibrage et documentation. Je n'ai **jamais pratiqué Lua**.

Partenaires externes : aucun. Le projet s'appuie sur le framework open-source **LÖVE** (love2d.org) et sa documentation.

Objectif fonctionnel

Lister les fonctionnalités majeures de chacune des parties du projet sous la forme de user story

https://fr.wikipedia.org/wiki/R%C3%A9cit_utilisateur

- En tant que joueur, je peux me déplacer au clavier (ZQSD ou flèches) sur une carte lunaire de 50×50 cases.
- En tant que joueur, je commence avec une barre d'oxygène pleine qui diminue de 1 point toutes les 30 secondes ; à 0, je meurs et la partie est perdue.
- Je peux consommer de la glace ou de l'oxygène pur pour survivre plus longtemps.
- En tant que joueur, mon champ de vision est limité et la carte est couverte de brouillard : je dois explorer pour découvrir le terrain, notamment pour voir les ressources.
- En tant que joueur, je subis un cycle jour/nuit.
- Je peux récolter différents minerais avec la touche F, à savoir : régolithe, glace, fer, titane, silicium et hélium-3.

- En tant que joueur, je vois la fusée en permanence, même dans le brouillard.
 - En tant que joueur, je peux réparer la fusée en 5 étapes en y déposant mes ressources avec la touche F, afin de gagner la partie (coque, réservoirs, électronique, moteurs et carburant).
 - En tant que joueur, mon score correspond au temps passé dans la partie – le plus court est le meilleur – et mon meilleur score est sauvegardé entre les parties.
 - Je peux ouvrir mon inventaire avec la touche E : ressources, oxygène, électricité, progression de l'objectif et objets fabricables sont affichés (indicateur vert/rouge).
 - Je peux lancer une fabrication, les ressources sont débitées tout de suite, mais l'objet n'est livré qu'après son temps de fabrication.
 - Je peux poser un objet en cliquant dessus dans l'inventaire puis sur une case valide de la carte.
-
- En tant que joueur, je peux fabriquer un **panneau solaire** (2 silicium + 1 fer, 5 sec) : 1 point d'électricité toutes les 5 sec le jour, +10 points de stockage.
 - En tant que joueur, je peux fabriquer une **batterie** (1 fer + 1 titane, 10 sec) : +50 points de stockage, partagé entre toutes les batteries de la carte.
 - En tant que joueur, je peux fabriquer un **électrolyseur** (2 fer + 1 titane, 10 sec) : transforme 1 glace + 6 électricité en 1 oxygène pur.

Environnement technique / technologique

Exposer le contexte technique et technologique (matériel, langage, environnement d'exécution, ressources, ...) dans lequel le projet s'inscrit.

Matériel :

- Un PC de développement sous Linux, capable de lancer le framework LÖVE.

Langage(s) :

- Lua 5.1

Framework :

- LÖVE (Love2D) 11.5

Environnement d'exécution :

- Runtime LÖVE
- Git

Ressources :

- Wiki officiel LÖVE (love2d.org/wiki)
- *Programming in Lua* (documentation de référence du langage)

Description du livrable

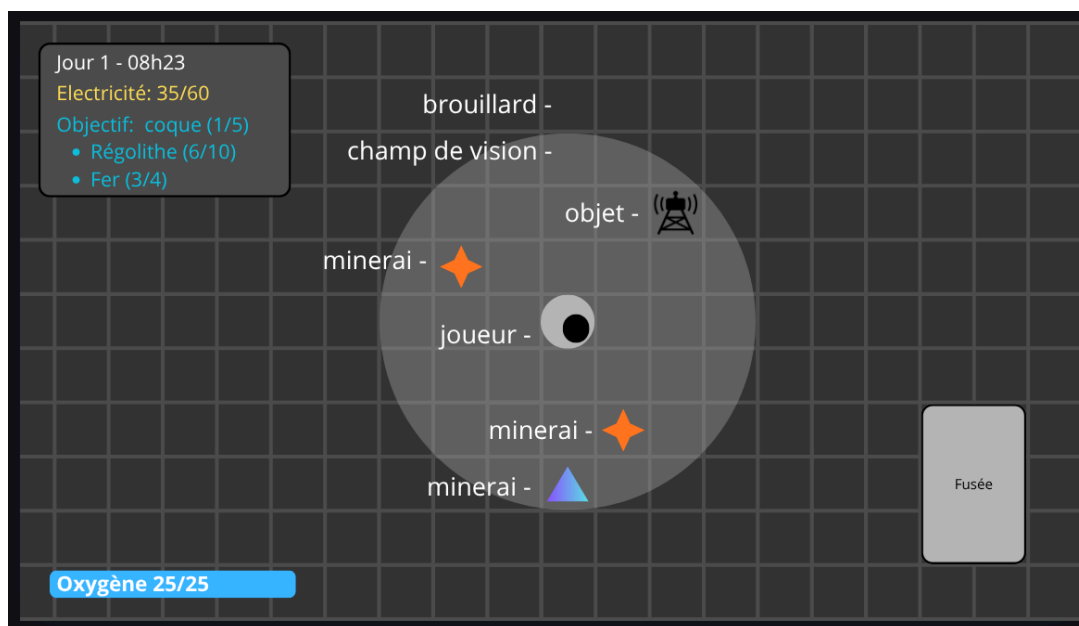
Détailler chaque élément (programmes, librairies, modules, assets, ...) du livrable et leur niveau de finition (déploiement, documentation, ...)

Le livrable sera un **jeu 2D complet jouable** avec un code source séparé en modules.

Il s'articule autour de quatre écrans :

- Un **menu d'accueil** : on y retrouve le titre, les contrôles, le meilleur score réalisé, et le bouton de lancement de la partie.
- Un **écran de jeu** : la carte est située au centre de l'écran (caméra, brouillard), et sur la gauche les informations liées au joueur et au monde (oxygène, heure, électricité, ...). On y trouve les différents minerais, les objets, et la fusée, nécessaire pour finir la partie.
- Un **écran d'inventaire / fabrication** (touche E) : l'inventaire affiche les ressources et la progression de l'objectif à gauche, les objets fabricables (vert/rouge, coût, durée) et les fabrications en cours à droite.
- Des **écrans de fin** : écran de victoire (score, record) ou de défaite (temps survécu, cause de défaite), avec un bouton pour relancer une partie ou revenir au menu.

Ci-dessous, un visuel de l'écran de jeu durant une partie (non définitif) :



Toutes les variables du jeu (coûts, délais, densités) seront centralisées dans un seul fichier de configuration, pour régler le jeu sans avoir à toucher au code. Le README servira à documenter l'installation, les contrôles, les règles et l'architecture.

Toutes les fonctionnalités seront opérationnelles et testées manuellement. L'équilibrage fin et le polissage visuel pourront rester perfectibles au rendu.

Organisation et temporalité

Exposer le plan de réalisation du projet : parties, dépendances, planification

Projet réalisé en solo, pour environ **8 à 9 jours de travail**. Les parties sont ordonnées du cœur de jeu vers le confort, et chaque partie sera dépendante de la précédente.

1 - Apprentissage de Lua et de LÖVE

Je commence par les fondamentaux du langage : la syntaxe, les tables et les modules. Je découvre ensuite le fonctionnement de LÖVE à travers sa boucle load / update / draw. Je réalise donc un mini-prototype jetable : une fenêtre, un carré déplaçable au clavier.

Temps estimé : 1 jour

2 - Architecture, carte et déplacements

Je découpe le code en modules, à raison d'un fichier par système, et je regroupe toutes les variables d'équilibrage dans un seul fichier de configuration. Je développe ensuite la génération de la carte de 50×50 cases, la caméra qui suit le joueur, et enfin les déplacements au clavier avec leurs collisions.

Temps estimé : 1 jour

3 - Exploration et ressources

La carte est couverte d'un brouillard de guerre, et le joueur ne voit qu'à l'intérieur de son champ de vision. Les ressources apparaissent aléatoirement sur la carte selon leurs densités, et le joueur peut les récolter avec la touche F pour les stocker dans un inventaire simple.

Temps estimé : 1 jour

4 - Survie : oxygène et cycle jour/nuit

J'ajoute ensuite la pression de survie : la barre d'oxygène diminue en continu, le joueur peut consommer de la glace pour la remonter, et il meurt si elle atteint zéro. Je développe aussi le cycle jour/nuit, avec l'heure affichée sous la forme « Jour 1 - 08h29 ».

Temps estimé : 1 jour

5 - La fusée et la boucle de victoire

Je place la fusée aléatoirement sur la carte, puis je développe les 5 étapes de réparation, dans lesquelles le joueur vient déposer ses ressources. Lorsque tout est réparé, le joueur peut décoller : la partie est gagnée et son score s'affiche.

Temps estimé : 1 jour

6 - Artisanat et pose d'objets

Je développe l'écran de fabrication : chaque objet affiche son coût avec un indicateur vert ou rouge, les ressources sont débitées au lancement de la fabrication et l'objet n'est livré qu'après son temps de fabrication. Le joueur peut ensuite poser ses objets au sol à la souris, en respectant les règles de placement (uniquement sur une case valide de la carte).

Temps estimé : 1 jour

7 - Le système d'électricité

Je termine le gameplay avec la chaîne de production : le panneau solaire produit de l'électricité le jour, la batterie la stocke (stockage partagé entre toutes les batteries de la carte), et l'électrolyseur consomme cette électricité avec de la glace pour produire de l'oxygène pur.

Temps estimé : 1 jour

8 - Interface finale, équilibrage et documentation

Création du menu d'accueil, des écrans de victoire et de défaite, du HUD complet et de la sauvegarde du meilleur score. Je termine par des sessions de jeu pour équilibrer les coûts et les délais, puis par la rédaction du README (dépendances et installation, contrôles, règles et architecture).

Temps estimé : 1 à 2 jours

Temps total estimé : 8 à 9 jours.